

Product Requirements Document

CPU-first autonomous railway safety agent

"Existing CCTV records incidents. RailGuard watches, remembers, reasons and acts before an incident becomes a disaster."

Document version	1.0 - Hackathon MVP and deployment handoff
Prepared for	Far Away 2026 - Round 1 / Railway Theme
Primary owner	Manish Sharma / CORTEX project team
Target reader	Engineer inheriting the current visual-memory MVP
Status	Build-ready specification; freeze P0 scope before implementation

CONFIDENTIAL BUILD DOCUMENT

Use this document to pivot the existing object-detection and visual-memory application into a complete perception-to-action railway safety MVP without rewriting the working foundation.

0 Document Purpose and Control

This PRD is a technical and product handoff. A developer familiar with the existing project should be able to identify reusable modules, add the missing railway-specific modules, run the system locally, demonstrate the complete workflow and package a deployable MVP. Where the current codebase differs from the recommended stack, preserve existing working code and implement the contracts defined here through adapters.

Scope discipline

The winning submission is not a universal agent for every hackathon theme. It is one focused, complete railway safety product with a reusable architecture. Build the closed loop - mission, perception, reasoning, memory, action and evidence - before adding optional features.

Table of Contents

Section	Contents
1	Executive Summary
2	Product Strategy and Success Definition
3	Existing MVP Assessment and Pivot Map
4	Users, Use Cases and User Journeys
5	Product Scope and Requirements
6	System Architecture
7	Detailed Component Specifications
8	Data Model and API Contracts
9	User Interface Specification
10	Integration Plan for the Current Codebase
11	Implementation Plan and Priorities
12	Testing, Acceptance and Demo Readiness
13	Deployment, Operations and Security
14	Risks, Limitations and Honest Claims
15	Submission and Demonstration Package
Appendix A	Reference Repository Structure
Appendix B	Configuration and Schema Examples
Appendix C	Core Algorithm Pseudocode
Appendix D	Definition of Done

Decision Log

ID	Decision	Rationale / outcome
D-01	Theme	Railways is the primary theme. Agentic autonomy is the enabling technology.
D-02	Product	CORTEX RailGuard: autonomous visual safety agent for existing railway CCTV.
D-03	Core demo	Platform-edge intrusion with severity escalation when a train is visible.
D-04	Architecture	Deterministic safety rules execute actions; natural language only configures missions.
D-05	Model	Reuse lightweight pretrained YOLOv8n; do not train a new detector before submission.
D-06	Deployment	CPU-first and local-first. Cloud or external integrations are optional adapters.
D-07	Evidence	Every answer and incident must be grounded in a timestamped frame or event record.

1 Executive Summary

CORTEX RailGuard transforms passive railway CCTV into an active safety system. An operator gives the system a monitoring mission such as “monitor the platform edge and create a critical incident if a person enters while a train is visible.” RailGuard then detects and tracks relevant objects, reasons about restricted zones and scene context, stores meaningful visual events, raises an alert, preserves evidence, creates a response task and generates a report. The system also answers constrained questions from its event memory and always returns supporting evidence.

The existing project already contains the foundation: lightweight object detection, foveal focus, object memory, locate-anything filtering and simple visual Q&A. The pivot is therefore an extension rather than a rewrite. The key additions are multi-object tracking, zone reasoning, a deterministic mission policy engine, event-level memory, action execution, evidence management and a railway-specific operator dashboard.

1	6	< 3 min	CPU
Focused product	P0 modules	Demo length	Primary runtime

1.1 Product Promise

Primary value proposition

Deploy on existing cameras and low-cost computers to convert continuous video into a small number of explainable, actionable and searchable safety incidents.

1.2 Winning Demonstration

1. Operator selects or enters a platform intrusion mission.
2. System processes a railway-platform video and tracks a person with a persistent ID.
3. The person enters the configured danger polygon.
4. RailGuard creates an incident, saves a snapshot and escalates severity because a train is visible.
5. Operator asks when the intrusion occurred and receives the timestamp plus evidence frame.
6. Operator generates a safety report showing the mission, incidents and actions taken.

1.3 Non-negotiable Product Principle

RailGuard must complete a visible perception-to-action loop. A detector that draws boxes is not the product. A chat interface that describes a scene is not the product. The product is the combination of mission, real-time monitoring, rule execution, evidence-backed event memory and action.

2 Product Strategy and Success Definition

2.1 Problem Statement

Railway operators may have many camera feeds, but video is primarily passive and operator attention is limited. Important events can be missed, searching previous footage is slow and the system often provides no structured incident, evidence package or response workflow. Small stations and lower-budget deployments also cannot assume continuous GPU or cloud access.

2.2 Target Outcome

A station or safety operator should be able to connect or upload a video, define a mission in less than one minute, receive a reliable event when a person enters a restricted zone, inspect the evidence, query the event history and export a report without manually reviewing the entire video.

2.3 Product Goals

Goal	Name	Measurable interpretation
G1	Complete closed loop	Mission -> perception -> reasoning -> event -> action -> report.
G2	Reuse current work	Preserve the existing detector, focus and memory interfaces wherever possible.
G3	CPU-first	Run the demonstration on the current laptop CPU without a mandatory GPU.
G4	Explainable	Every incident includes the rule that fired, object ID, timestamp and evidence.
G5	Operator useful	Show relevant objects and incidents instead of all detections.
G6	Extensible	Permit new missions, zones and action adapters without changing the detector.

2.4 Success Metrics

Metric	Target
Mission setup time	<= 60 seconds using preset or form-based mission
Incident creation latency	<= 1.5 seconds after rule becomes true in processed time
Duplicate alerts	At most one incident per track per rule during configured cooldown
Evidence completeness	100% of incidents include timestamp, object ID, rule and snapshot
Q&A grounding	100% of supported queries return an event reference or "no matching event"
Demo reliability	Five consecutive end-to-end runs without manual code changes
Runtime target	>= 8 FPS on a common laptop CPU at 640-pixel inference; >= 4 FPS is acceptable fallback
Startup	Single command or documented two-command startup

2.5 North-star Narrative

“RailGuard does not try to understand everything in every frame. It focuses on a defined safety mission, watches the relevant objects, remembers only meaningful events and performs a controlled action with evidence.”

3 Existing MVP Assessment and Pivot Map

3.1 Current Capabilities to Preserve

Existing capability	Pivot use
Lightweight object detection	Reuse model loading, preprocessing, inference and bounding-box rendering.
Foveal attention / crop focus	Reuse to enlarge the mission-relevant object or evidence frame.
Visual memory across images or frames	Reuse storage/query patterns, but change the primary unit from raw object occurrence to structured event.
Locate-anything command	Reuse as a UI filter and evidence retrieval function.
Simple visual Q&A	Reuse command routing and response UI, but answer from event memory rather than free-form guesses.
CPU-first runtime	Preserve lightweight inference and configurable frame skipping.

3.2 Why the Current Product Is Not Yet Sufficient

- No persistent tracking identity, so the same person may be treated as a new object in every frame.
- No railway safety geometry such as platform-edge polygons or track regions.
- Memory stores detections rather than incidents and actions.
- No deterministic rule engine that can explain why an alert fired.

- No autonomous action beyond displaying or answering.
- No operator incident timeline or report.
- Broad claims such as “locate anything” are limited by COCO classes and should be presented honestly.

3.3 Pivot Mapping

Module	Decision	Required change
Detector	Keep	Wrap output into a standard Detection schema.
Frame processor	Keep / modify	Add frame ID, video timestamp, scene context and configurable frame stride.
Foveal attention	Keep	Use for selected track and incident evidence preview.
Object memory	Refactor	Create EventStore; optionally retain raw detections only for debugging.
Locate command	Keep / narrow	Support known classes and event-backed filters.
VQA	Refactor	Use a supported intent parser and SQL/event queries; no unsupported free-form claims.
UI	Modify	Add mission panel, zone editor/preset, incident timeline, action log and report.
New: tracker	Add	ByteTrack or existing tracker adapter.
New: zone engine	Add	Point-in-polygon and entry/exit state.
New: policy engine	Add	Mission rules, severity and cooldown.
New: action executor	Add	Snapshot, alert, response task, webhook and report adapter.
New: report service	Add	HTML/Markdown first; PDF optional.

Do not rewrite the detector

A clean adapter around the current inference output is safer than replacing working detection code. Implement the new system around stable interfaces, then improve internals after the submission.

4 Users, Use Cases and User Journeys

4.1 Primary Personas

Persona	Needs
Station safety operator	Configures monitoring mission, watches alerts, acknowledges incidents and exports report.
Security supervisor	Reviews incident timeline, evidence and response actions after the event.
Deployment technician	Connects video source, draws zones, configures model/runtime and validates camera coverage.

Persona	Needs
System administrator	Manages users, retention, integrations and health in a future production deployment.

4.2 P0 Use Case: Platform-edge Intrusion

An operator selects “Platform-edge intrusion,” chooses or draws the restricted polygon and starts monitoring. When a tracked person’s foot point enters the polygon, the system creates one incident. If a train is currently visible, severity is CRITICAL; otherwise it is HIGH. RailGuard saves the evidence frame, emits a visible/audio alert, creates a response task and records all actions.

4.3 P1 Use Cases

Use case	Trigger	Expected action
Crowd threshold	Person count in zone exceeds configured threshold for N seconds	Create crowding incident and periodic update
Unattended baggage	Bag remains stationary and no associated person remains nearby for threshold duration	Create suspicious baggage incident
Track trespass	Person enters track polygon	Immediate critical alert
Gate obstruction	Object remains in gate zone	Create operational warning
Retrospective search	Operator asks for last known event involving class/track	Return time, location and snapshot

4.4 Golden User Journey

Stage	User/system experience
1. Configure	Choose source, platform-edge preset and severity rules.
2. Calibrate	Draw or load polygon; run 10-second preview.
3. Monitor	Start mission; interface highlights only mission-relevant tracks.
4. Detect	Policy engine identifies intrusion and scene context.
5. Act	Save evidence, alert operator and create response task.
6. Investigate	Open incident, inspect evidence and ask a supported memory question.
7. Close	Acknowledge incident and export safety report.

5 Product Scope and Requirements

5.1 Scope Priorities

Priority	Meaning	Rule
----------	---------	------

P0	Required for submission	Must work end-to-end before recording.
P1	Competitive enhancement	Add only after all P0 acceptance tests pass.
P2	Post-hackathon roadmap	Document, but do not risk the submission.

5.2 Functional Requirements - P0

ID	Area	Requirement	Pri.
FR-001	Video source	Accept MP4/MOV upload. Optional webcam/RTSP adapter.	P0
FR-002	Detection	Detect person and train; detect suitcase/backpack when available.	P0
FR-003	Tracking	Assign stable track IDs across frames using ByteTrack or equivalent.	P0
FR-004	Zone	Load or draw one or more named polygons and persist normalized coordinates.	P0
FR-005	Mission	Select preset or enter constrained natural-language mission.	P0
FR-006	Policy compile	Convert mission to deterministic Rule objects; show compiled rule to user.	P0
FR-007	Intrusion	Trigger when tracked person foot point transitions from outside to inside zone.	P0
FR-008	Severity	Escalate to CRITICAL if train_visible=true during intrusion.	P0
FR-009	Deduplication	Prevent repeated alerts for the same track and rule during cooldown.	P0
FR-010	Evidence	Save annotated and clean frame crops plus metadata for every incident.	P0
FR-011	Actions	Create visual alert, local response task and action-log entry.	P0
FR-012	Timeline	Display ordered incident cards with time, severity, rule and evidence.	P0
FR-013	Memory query	Support at least five deterministic event-memory questions.	P0
FR-014	Report	Generate HTML or Markdown incident report; PDF optional.	P0
FR-015	Config	Provide a single configuration file for thresholds, model and paths.	P0
FR-016	Demo mode	Include a known video, known zone and one-click mission preset.	P0

5.3 Functional Requirements - P1/P2

ID	Feature	Priority	Notes
FR-101	Crowd threshold mission	P1	Count unique active tracks in a polygon for a sustained duration.
FR-102	Webhook action	P1	POST signed incident payload to configured URL.
FR-103	Email draft	P1	Generate response email; sending requires explicit operator action.
FR-104	Unattended baggage	P1	Stationary bag plus distance/association heuristic.
FR-105	Multi-zone calibration	P1	Named platform, track and gate polygons.
FR-201	Multi-camera identity	P2	Re-identification across cameras.
FR-202	Custom railway model	P2	Fine-tune on rail-specific data and edge classes.
FR-203	Role-based access	P2	Operator, supervisor and administrator permissions.
FR-204	Fleet deployment console	P2	Remote health and model/config rollout.

5.4 Non-functional Requirements

ID	Quality	Requirement
NFR-001	Reliability	No crash on unreadable frame; skip and log.
NFR-002	Performance	Target ≥ 8 FPS CPU at 640 input. Frame stride configurable.
NFR-003	Explainability	Store condition values and rule ID with event.
NFR-004	Privacy	Local processing by default; no video upload to external service.
NFR-005	Portability	Linux first; Windows-compatible Python environment desirable.
NFR-006	Reproducibility	Pinned requirements and sample configuration.
NFR-007	Graceful degradation	If tracking fails, continue detection and show degraded status.
NFR-008	Auditability	Actions append to immutable-style event/action log.
NFR-009	Accessibility	Severity conveyed with text/icon, not color alone.
NFR-010	Honesty	Clearly label simulated integrations and supported classes.

5.5 Out of Scope for MVP

- Automatic physical train control or braking.
- Face recognition, identity inference or biometric profiling.
- Guaranteed detection of objects outside the model class set.
- Legal or safety certification.
- Autonomous emergency dispatch to public authorities.
- Production-scale multi-camera re-identification.
- Claims of state-of-the-art model accuracy.

6 System Architecture

The architecture is intentionally modular. Existing modules can remain in place if they produce or consume the contracts below. The mission layer must not directly depend on YOLO objects, and the UI must not directly query detector internals. This separation makes the current project easier to pivot and later replace individual components.

CORTEX RailGuard - Reference Architecture

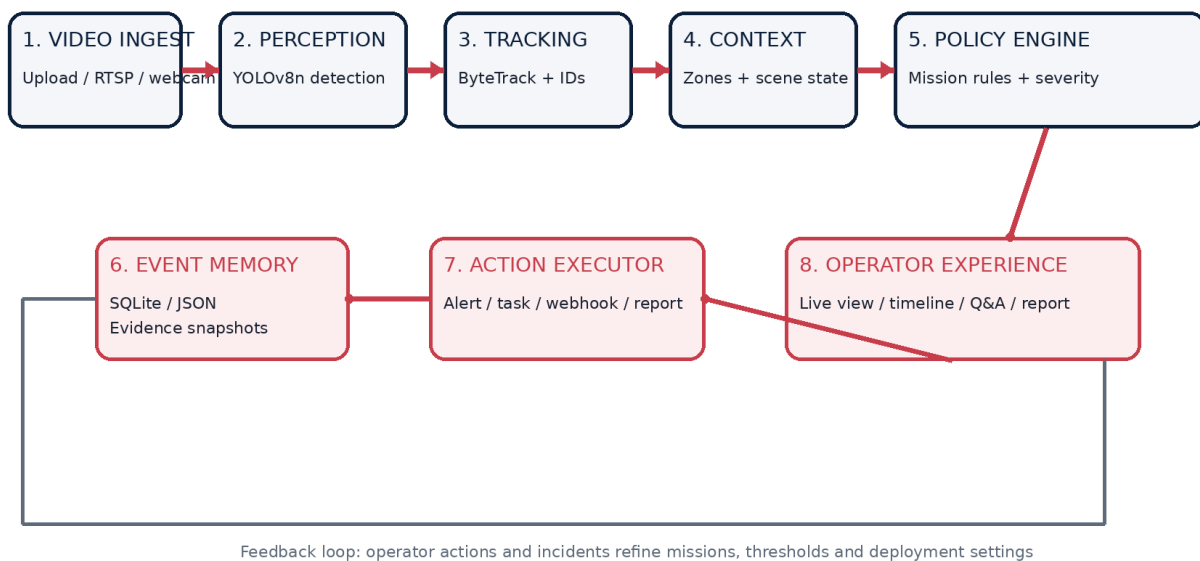


Figure 1. Reference perception-to-action architecture.

6.1 Processing Flow

Step	Component	Output
A	Ingest	Decode frame, generate frame_id and timestamp_ms.
B	Detect	Return Detection[] with class, confidence and bbox.
C	Track	Associate detections, return Track[] with

Step	Component	Output
		stable IDs.
D	Context	Calculate zone membership, train visibility, counts and object motion.
E	Evaluate	Apply deterministic rules to current context and prior track state.
F	Create event	Persist one event on state transition; attach evidence.
G	Execute actions	Alert, task, webhook adapter and report availability.
H	Serve UI/Q&A	Stream status and query EventStore.

6.2 Deployment Topology

For the hackathon, the recommended topology is a single local process or two local processes: a Python inference/service layer and the existing UI. A database server is not required; SQLite is sufficient. For deployment, the same modules can run in containers, with video sources and evidence mounted as volumes.

Recommended MVP topology

```

[Camera / Video File]
  |
  v
[RailGuard Python Service] -- SQLite + evidence/ directory
  |
  +-- REST / WebSocket --> [Existing UI or Streamlit]
  |
  +-- Optional webhook --> [Mock station response system]
```

6.3 Trust Boundary

Safety decisions are deterministic. Natural-language mission parsing may select a predefined policy template and fill parameters, but the compiled rule is shown before execution. No language model response directly triggers an external action. Unsupported mission text must return a validation error or request the user to select a preset.

7 Detailed Component Specifications

7.1 Video Ingest Service

Responsibilities: open uploaded file or configured stream, normalize frames to BGR/RGB as required, derive accurate timestamps, handle end-of-stream and provide backpressure. The video service owns playback state; detector code receives FramePacket objects rather than reading the video directly.

Input	Output	Failure behavior
Path / source URI / camera index	FramePacket(frame_id, timestamp_ms, image, source_id)	Emit health error, do not crash UI.
Config: stride, resize, start/end	Ordered packets	Skip corrupt frames with warning.

Input	Output	Failure behavior
Stop/cancel token	Clean stream termination	Release capture handle in finally block.

7.2 Detector Adapter

Wrap the current YOLOv8n output into a model-independent Detection schema. Filter classes only after inference unless the current library supports class filtering efficiently. Keep a configurable confidence threshold; use a lower threshold only if the known demo video requires it.

Detector contract

```
class Detection(TypedDict):
    class_id: int
    class_name: str
    confidence: float
    bbox_xyxy: tuple[float, float, float, float]

class Detector(Protocol):
    def infer(self, frame: np.ndarray) -> list[Detection]: ...
```

7.3 Tracking Service

Use Ultralytics ByteTrack integration when compatible with the current detector. Otherwise feed detections to a standalone ByteTrack implementation. The tracker must return track_id, class, latest box, confidence and age. A lost track remains eligible for a short buffer but must not create new incidents while absent.

Parameter	Default	Reason
tracker	bytetrack.yaml	Fast, no appearance model required.
track_buffer	30 frames	Tolerates short occlusion.
match_thresh	0.8	Stable demo IDs; tune only if needed.
classes	person, train, suitcase, backpack	Mission relevance.
persist	true	Maintain IDs across frames.

7.4 Zone Manager

Zones are polygons stored as normalized coordinates from 0 to 1 so they survive resolution changes. Each zone has an ID, display name, zone type and severity modifier. The person location is represented by the bottom-center point of the bounding box because it approximates contact with the ground plane better than the box center.

Point-in-polygon rule

```
foot_x = (x1 + x2) / 2
foot_y = y2
inside = cv2.pointPolygonTest(zone_polygon_px, (foot_x, foot_y), False) >= 0
```

7.5 Mission and Policy Engine

A Mission contains one or more Rules and Actions. For the P0 demonstration, provide preset templates and optionally a constrained parser. The parser maps known phrases to parameters. It must never invent an unsupported detector class or action.

Mission field	Example
mission_id	platform_intrusion_v1
objects	[person, train]
primary_zone	platform_edge
trigger	person enters platform_edge
severity_base	HIGH
escalation	if train_visible then CRITICAL
cooldown_seconds	5
actions	[save_evidence, alert_ui, create_task]

7.6 Event Engine and Deduplication

Events are created on a state transition, not continuously while a condition remains true. The event engine stores per-track and per-rule state. The recommended state machine is CLEAR -> OBSERVE -> INCIDENT -> ACTIONED -> COOLDOWN -> CLEAR. A new event may be created only after the object exits the zone and the cooldown expires, or under an explicit repeat policy.

Intrusion Event State Machine

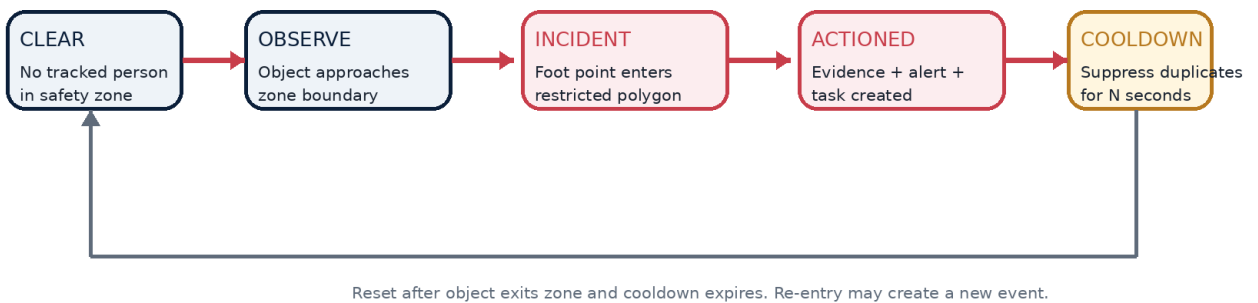


Figure 2. Intrusion event state and duplicate suppression.

7.7 Event Memory

The memory layer becomes an EventStore. It should store high-value facts and evidence rather than every raw box. Raw detections may be kept in an optional rolling buffer for debugging. Event queries must be deterministic and return a typed result.

Supported intent	Example question	Query behavior
LIST_INCIDENTS	What incidents occurred?	List events ordered by timestamp.
MOST_CRITICAL	Show the most critical incident.	Sort by severity then time.
LAST_SEEN	When was person 7 last seen?	Latest event or observation for track 7.
COUNT_ZONE_ENTRIES	How many people entered the restricted zone?	Count unique intrusion events.
TRAIN_CONTEXT	Was a train visible during the incident?	Read scene_context.train_visible.
NO_MATCH	Where was an elephant?	Return unsupported class/no matching evidence.

7.8 Evidence Service

For each event, save an annotated full frame and an object/zone crop where useful. Use filenames containing event ID and timestamp, not user-entered text. Store paths relative to the project data directory. If storage fails, still create the event with evidence_status=FAILED and show the failure in the UI.

7.9 Action Executor

Actions are adapters with a common result. The P0 actions are local and real: save evidence, show alert, play sound and create an internal response task. A webhook action is optional. Simulated actions must be labelled “demo integration” rather than presented as a live railway system connection.

Action adapter contract

```
class ActionResult(TypedDict):
    action_type: str
    status: Literal['SUCCESS', 'FAILED', 'SKIPPED']
    executed_at: str
    detail: str

class ActionAdapter(Protocol):
    def execute(self, event: IncidentEvent) -> ActionResult: ...
```

7.10 Report Service

Generate an HTML or Markdown report from the database. PDF conversion is optional and should not block the MVP. The report must include mission configuration, source information, run time, total incidents by severity, incident table, actions, evidence thumbnails and limitations.

8 Data Model and API Contracts

8.1 Core Entities

Entity	Purpose
Mission	Configuration that states what to watch, where and what to do.
Zone	Normalized polygon and semantic type.
Track	Current identity and motion state for a detected object.
SceneContext	Derived facts for one frame: visible classes, counts and zone

Entity	Purpose
	membership.
IncidentEvent	Persisted safety event created by a rule transition.
ActionLog	Result of each attempted action.
ResponseTask	Operator-facing task generated from an incident.
SystemHealth	Runtime FPS, model status, source status and errors.

8.2 Incident Schema

Canonical incident JSON

```
{
  "event_id": "RG-20260614-0001",
  "mission_id": "platform_intrusion_v1",
  "rule_id": "person_enters_platform_edge",
  "source_id": "demo_platform_01",
  "frame_id": 542,
  "timestamp_ms": 18066,
  "occurred_at": "2026-06-14T15:42:17+05:30",
  "track_id": 7,
  "object_class": "person",
  "zone_id": "platform_edge",
  "event_type": "RESTRICTED_ZONE_INTRUSION",
  "severity": "CRITICAL",
  "condition_values": {
    "inside_zone": true,
    "train_visible": true,
    "person_count_in_zone": 1
  },
  "evidence": {
    "annotated_frame": "evidence/RG-20260614-0001.jpg",
    "object_crop": "evidence/RG-20260614-0001-crop.jpg"
  },
  "status": "OPEN"
}
```

8.3 SQLite Tables

Minimum database schema

```
CREATE TABLE incidents (
  event_id TEXT PRIMARY KEY,
  mission_id TEXT NOT NULL,
  rule_id TEXT NOT NULL,
  source_id TEXT NOT NULL,
  frame_id INTEGER NOT NULL,
  timestamp_ms INTEGER NOT NULL,
  occurred_at TEXT NOT NULL,
  track_id INTEGER,
  object_class TEXT,
  zone_id TEXT,
  event_type TEXT NOT NULL,
  severity TEXT NOT NULL,
  condition_json TEXT NOT NULL,
  evidence_json TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'OPEN'
);

CREATE TABLE action_logs (
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT,
event_id TEXT NOT NULL,
action_type TEXT NOT NULL,
status TEXT NOT NULL,
executed_at TEXT NOT NULL,
detail TEXT,
FOREIGN KEY(event_id) REFERENCES incidents(event_id)
);
```

8.4 Recommended REST API

Method	Path	Purpose
POST	/api/sources/upload	Upload and register demo video.
POST	/api/missions/compile	Validate preset/text and return compiled mission.
POST	/api/runs	Start monitoring run.
POST	/api/runs/{id}/stop	Stop run and flush state.
GET	/api/runs/{id}/status	Health, frame, FPS and current mission.
GET	/api/incidents	Filter by run, severity, zone, class or time.
GET	/api/incidents/{event_id}	Incident, evidence and action logs.
POST	/api/incidents/{event_id}/ack	Acknowledge or close incident.
POST	/api/query	Execute supported memory query.
GET	/api/reports/{run_id}	Generate/download report.
WS	/ws/runs/{id}	Stream frame/metadata and incidents to UI.

8.5 WebSocket Event Types

Type	Payload
frame	frame_id, timestamp_ms, optional JPEG, tracks, zones
incident_created	full IncidentEvent summary
action_result	event_id and ActionResult
health	fps, queue depth, detector/tracker/source status
run_finished	run_id, counts and report URL
error	code, human message and recoverability

9 User Interface Specification

9.1 Screen Layout

Retain the existing UI framework. The recommended layout is a two-column operator dashboard: live/recorded video on the left and mission plus incidents on the right. A bottom panel contains memory query and report actions. The interface must be usable during a recorded demo without scrolling through technical controls.

Area	Required elements
Header	Product name, run status, source, CPU/FPS indicator.
Mission panel	Preset selector, text field, compiled rule preview, Start/Stop.
Video panel	Frame, zone overlay, selected tracks, timestamp and severity banner.
Incident timeline	Newest first cards with severity text, time, rule, snapshot and status.
Incident drawer	Evidence, condition values, action log and acknowledge button.
Memory panel	Five example queries plus typed input.
Report panel	Run summary and Generate Report button.
Health panel	Source/model/tracker/database status; hidden by default in demo.

9.2 Visual Rules

- Do not show every detection box. Show mission-relevant tracks and selected evidence.
- Use both label and color for severity: CRITICAL, HIGH, MEDIUM.
- Display the compiled mission before Start so the system appears controlled, not magical.
- When an incident fires, freeze or spotlight the evidence for 1-2 seconds without stopping processing.
- All memory answers include “Based on incident RG-...” and a thumbnail.
- Use a DEMO label for simulated task dispatch or webhook destinations.

9.3 Mission Presets

Preset	Default text	Configuration
Platform intrusion	Monitor the platform edge. Critical if a train is visible.	person + platform_edge + train escalation
Track trespass	Alert immediately if a person enters the track zone.	person + track_zone + CRITICAL
Crowd watch	Alert when more than 8 people remain in the waiting zone for 10 seconds.	person count + duration

9.4 Error and Empty States

Condition	User message / behavior
No model file	Model unavailable. Check path in config; monitoring not started.
Unsupported mission	This MVP supports platform intrusion, track trespass and crowd watch. Select a preset.
No incident	No matching incident found in this run.
Evidence write failure	Incident stored, but evidence could not be written. Check storage path.
Low FPS	Performance degraded; frame stride increased to maintain responsiveness.
Source ended	Monitoring complete. Generate report or replay.

10 Integration Plan for the Current Codebase

10.1 Integration Philosophy

The developer should first make a dependency map of the current app, then introduce interfaces at the edges. Do not move files or rename working modules until the end-to-end path runs. The fastest safe pivot is to keep the current inference loop and progressively replace side effects with calls to new services.

10.2 First 30-Minute Codebase Audit

- 7. Find the main entrypoint and exact command used to start the current app.
- 8. Locate model loading and identify whether the model is loaded once or per request.
- 9. Locate the per-frame inference loop and output format.
- 10. Locate visual-memory storage and query routing.
- 11. Locate UI components for upload, frame display and questions.
- 12. Create a branch named railguard-pivot and tag the last working version.
- 13. Run and record a baseline test before changing code.

10.3 Adapter-by-Adapter Pivot

Order	Module	Change
Step 1	Detection adapter	Translate current detection objects into Detection dictionaries. No behavior change.
Step 2	FramePacket	Add frame_id/timestamp/source_id around existing frame loop.
Step 3	Tracker	Attach track IDs. Verify IDs on a 20-second clip.
Step 4	Zone manager	Hard-code demo polygon first; add editor later.
Step 5	Context builder	Derive train_visible and per-track zone membership.
Step 6	Policy engine	Implement one hard-coded mission, then load from YAML.
Step 7	EventStore	Write incidents and evidence; keep old object memory temporarily.
Step 8	Actions	Connect incident callback to alert/task/log.
Step 9	Memory Q&A	Route supported questions to EventStore.
Step 10	UI	Replace generic object panels with mission/timeline/report.
Step 11	Cleanup	Remove or hide legacy features that confuse the demo.

10.4 Recommended Compatibility Layer

Compatibility module pattern

```
# railguard/adapters/current_project.py
```

```
def normalize_current_detections(raw_results) -> list[Detection]:
    # Only this function knows the current detector output format.
    ...

def write_legacy_memory(event: IncidentEvent) -> None:
    # Optional bridge while the old visual memory remains in use.
    ...

def render_with_current_ui(frame, tracks, zones, incidents):
    # Preserve existing UI while new panels are integrated.
    ...
```

10.5 UI-framework Decision

Current UI	Recommendation
Streamlit	Keep it. Use session_state, placeholders and a background worker/queue.
Gradio	Keep it. Use generator/stream callbacks and state objects.
React / Next.js	Keep frontend; expose FastAPI REST/WebSocket service.
OpenCV desktop window	Keep for inference demo, add a lightweight Streamlit incident dashboard if time permits.
Notebook	Move core pipeline into Python modules; create a minimal Streamlit runner.

Critical integration rule

Model loading, tracker state and database connection must be long-lived. Do not recreate them for each frame or UI rerender.

11 Implementation Plan and Priorities

11.1 Emergency Build Sequence

Elapsed	Deliverable
0:00-0:20	Freeze scope, branch, baseline run and choose demo video.
0:20-1:20	Detection adapter + tracking IDs.
1:20-2:05	Hard-coded/preset zone + foot-point entry.
2:05-3:00	Policy, severity and duplicate suppression.
3:00-4:00	SQLite/JSON EventStore + evidence snapshots.
4:00-4:45	Alert, response task and action log.
4:45-5:30	Memory query intents.
5:30-6:15	Dashboard timeline and incident view.
6:15-6:45	HTML/Markdown report.

Elapsed	Deliverable
6:45-7:30	Five-run test, README, screenshots and demo recording.

11.2 Build Gates

Gate	Pass condition
Gate A	Person retains a track ID for $\geq 80\%$ of visible sequence.
Gate B	Entering the polygon fires exactly one incident.
Gate C	Incident includes snapshot and appears in timeline.
Gate D	Train visibility changes severity to CRITICAL.
Gate E	Three memory queries return evidence-backed answers.
Gate F	Report is generated from stored data.
Gate G	End-to-end demo works five times consecutively.

11.3 Fallback Ladder

Failure	Fallback without losing the product story
Natural-language parser unstable	Use mission presets and editable form fields.
ByteTrack integration breaks	Use centroid/IOU tracker for demo clip; disclose simplified tracker.
Live stream unstable	Use uploaded MP4 as primary demo.
PDF export fails	Deliver HTML or Markdown report.
Audio alert unavailable	Use visible banner and logged response task.
FPS too low	Reduce inference size, process every second frame, display measured FPS.
Train detector misses train	Choose a suitable demo clip or use scene preset only if honestly labelled; do not fake detection.
Zone editor consumes time	Ship a preset JSON polygon with an optional coordinate form.

11.4 Post-submission Hardening

- Add unit tests and typed Pydantic models.
- Replace hard-coded paths with configuration and environment variables.
- Add authentication and retention controls.
- Benchmark multiple CPU profiles and quantized models.
- Collect railway-specific dataset and calibrate false alarms.
- Validate with railway domain experts before any real deployment.

12 Testing, Acceptance and Demo Readiness

12.1 Unit Test Matrix

ID	Test target	Expected result
UT-01	point_in_polygon	Inside, boundary and outside points.
UT-02	foot_point	Correct bottom-center coordinate.
UT-03	severity	Train visible escalates HIGH -> CRITICAL.
UT-04	dedupe	Same track does not repeat inside cooldown.
UT-05	reentry	Exit + cooldown + re-entry creates new event.
UT-06	event serialization	JSON/SQLite round-trip.
UT-07	query parser	Five supported intents and unsupported input.
UT-08	action result	Failure is logged without crashing pipeline.
UT-09	normalized zone	Polygon scales correctly to two resolutions.

12.2 End-to-End Acceptance Scenarios

ID	Scenario	Pass criterion
AC-01	Start demo preset	Mission compiles and run starts without code edit.
AC-02	No intrusion	Person outside zone does not create incident.
AC-03	Intrusion without train	One HIGH incident.
AC-04	Intrusion with train	One CRITICAL incident and all P0 actions.
AC-05	Duplicate suppression	No second incident while same track remains inside.
AC-06	Memory query	Question returns correct event and evidence.
AC-07	Report	Report count equals database count.
AC-08	Restart	Application can restart and load prior run incidents.
AC-09	Bad source	Clear recoverable error.
AC-10	Five-run reliability	Five consecutive successful demos.

12.3 Demo Dataset Checklist

- Video license or source is documented.
- Person clearly crosses a visible boundary.
- Train is visible at or near the intrusion moment.
- Lighting and resolution are sufficient for pretrained detector.

- Demo clip is 20-60 seconds and small enough for fast startup.
- Known zone coordinates are saved in demo config.
- Expected event timestamp is recorded for validation.

12.4 Performance Test

Log end-to-end FPS, detector time, tracking time, rule time and queue depth. Performance claims in the presentation must use measured values from the demo machine. If frame stride is used, disclose it as “processed at X FPS; video displayed at Y FPS.”

13 Deployment, Operations and Security

13.1 Local Setup

Reference startup commands

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp config.example.yaml config.yaml
python -m railguard.db.migrate
python -m railguard.app --config config.yaml
```

13.2 Minimum Requirements

Item	Hackathon MVP
OS	Ubuntu/Linux Mint preferred; Windows supported where current app works.
Python	3.10 or 3.11 recommended.
CPU	4 cores minimum; 6+ preferred.
RAM	8 GB minimum.
GPU	Not required; optional CUDA acceleration.
Storage	2 GB plus video/evidence retention.
Camera	Uploaded video required; webcam/RTSP optional.

13.3 Configuration

All runtime choices must be configurable: model path, device, image size, confidence, frame stride, tracker, database path, evidence path, cooldown, zones, source and action adapters. Secrets for webhook/email must come from environment variables, never the repository.

13.4 Containerization

A Dockerfile is desirable but not required before the working demo. The container should mount /data for database, evidence and uploaded video. Avoid bundling large model files in Git; download through a documented setup step or use a release asset where allowed.

13.5 Security and Privacy

- Process locally by default and store only event evidence required for the demo.

- Do not implement face recognition or infer sensitive identity attributes.
- Validate uploaded file type and cap file size.
- Sanitize filenames and never execute user-supplied paths.
- Use parameterized SQL or ORM queries.
- Sign webhook payloads in later deployments.
- Add configurable evidence retention and deletion in production.
- Display that the system is a prototype and not safety-certified.

13.6 Observability

Signal	Minimum implementation
Structured logs	Timestamp, run_id, frame_id, event_id, module and error.
Health	Source, detector, tracker, database and evidence path status.
Performance	Rolling FPS and component latency.
Event audit	Rule condition values and action results.
Demo diagnostics	Button or command to export logs/config for debugging.

14 Risks, Limitations and Honest Claims

14.1 Key Risks

ID	Risk	Mitigation	Impact
R1	Detector misses small/occluded people	Choose suitable demo; tune threshold; document limitation.	High
R2	Track IDs switch	Tune tracker; use short clip and clear motion.	Medium
R3	False intrusion from bbox jitter	Use foot-point hysteresis or require 2 consecutive frames.	Medium
R4	UI blocks inference	Move pipeline to worker thread/process; use queue.	High
R5	Overclaiming autonomy	Show deterministic compiled mission and action log.	High
R6	Unstable external integrations	Use internal response task as P0; webhook only as optional.	Medium
R7	Time spent on polish before core works	Enforce build gates and fallback ladder.	High

14.2 Claims We Can Make

- “CPU-first prototype that works with existing video sources.”
- “Combines detection, tracking, spatial reasoning, event memory and autonomous local actions.”
- “Stores timestamped evidence and answers supported questions from event memory.”
- “Designed as a reusable mission-driven architecture for rail and other infrastructure.”

14.3 Claims We Must Not Make

- “Prevents all railway accidents.”

- “State-of-the-art accuracy.”
- “Understands any object or any question.”
- “Production-ready for live railways without validation.”
- “Connected to railway emergency systems” unless a real integration exists.

Judge trust matters

A precise prototype with visible evidence and acknowledged limits is stronger than a broad system making unverifiable claims.

15 Submission and Demonstration Package

15.1 Required Repository Contents

Artifact	Expectation
README.md	Problem, solution, architecture, setup, demo steps, limitations.
requirements.txt / pyproject.toml	Pinned runtime dependencies.
config.example.yaml	All non-secret settings.
src/railguard/	Modular application code.
tests/	Core rule and state tests.
demo/	Small config, zone and link/instructions for video.
docs/architecture.png	Architecture diagram.
LICENSE / THIRD_PARTY.md	License and model/tool attribution.
.gitignore	Exclude evidence, database, model weights and secrets.

15.2 Two-to-Three-Minute Demo Sequence

Time	Beat	What to show/say
0:00-0:15	Problem	Existing CCTV records but does not create actionable, searchable safety events.
0:15-0:30	Mission	Select platform intrusion; show compiled rule.
0:30-1:15	Monitoring	Play clip; show tracking, zone and critical incident.
1:15-1:40	Memory	Ask when it happened and show evidence.
1:40-2:00	Action	Open response task and action log.
2:00-2:20	Report	Generate report.
2:20-2:40	Scale	CPU-first, existing cameras, reusable mission engine.
2:40-2:50	Close	“CCTV tells us what happened. RailGuard helps act before what happens next.”

15.3 Presentation Outline

- 14. Railway safety and passive CCTV problem.
- 15. CORTEX RailGuard product promise.
- 16. Working end-to-end demo.
- 17. Technical architecture and deterministic safety control.
- 18. CPU-first deployment and existing-camera economics.
- 19. Impact, roadmap and honest limitations.

15.4 Judge Q&A Preparation

Question	Answer
Is this just YOLO?	YOLO is one replaceable perception component. The product adds tracking, spatial/temporal policy execution, event memory, evidence and actions.
Where is the agent?	The mission agent converts an operator goal into a validated policy and autonomously monitors, evaluates, records and executes approved actions.
Why railways?	Existing cameras, high consequence of missed events, need for local processing and a globally relevant deployment pattern.
How do you avoid hallucinations?	Safety decisions are deterministic. Q&A reads structured events and returns evidence.
Can it run without cloud?	Yes, the core workflow is local and CPU-first.
What comes next?	Rail-specific model tuning, multi-camera deployment, expert validation and certified integrations.

Appendix A Reference Repository Structure

Suggested modular structure

```
cortex-railguard/  
├── README.md  
├── pyproject.toml  
├── config.example.yaml  
├── .env.example  
├── src/railguard/  
│   ├── app.py  
│   ├── config.py  
│   ├── domain/  
│   │   ├── models.py  
│   │   └── enums.py  
│   ├── ingest/video_source.py  
│   ├── perception/detector.py  
│   ├── perception/current_detector_adapter.py  
│   ├── tracking/tracker.py  
│   ├── context/zones.py  
│   ├── context/scene_builder.py  
│   ├── policy/mission_compiler.py  
│   ├── policy/rule_engine.py  
│   ├── events/event_engine.py  
│   ├── memory/event_store.py  
│   ├── evidence/evidence_service.py  
│   ├── actions/  
│   │   ├── base.py  
│   │   ├── ui_alert.py  
│   │   ├── response_task.py  
│   │   └── webhook.py
```

```
|   ├── query/query_service.py
|   ├── reports/report_service.py
|   ├── api/routes.py
|   ├── ui/ # only if UI is Python-based
|   ├── tests/
|   ├── demo/
|   |   ├── mission.yaml
|   |   ├── zones.json
|   |   └── README.md
|   ├── data/ # gitignored
|   |   ├── railguard.db
|   |   ├── evidence/
|   |   └── reports/
```

Appendix B Configuration and Schema Examples

config.example.yaml

```
app:
  demo_mode: true
  data_dir: ./data

model:
  name: yolov8n.pt
  device: cpu
  image_size: 640
  confidence: 0.35
  classes: [person, train, suitcase, backpack]

video:
  frame_stride: 1
  max_queue: 4

tracker:
  type: bytetrack
  config: bytetrack.yaml

policy:
  cooldown_seconds: 5
  consecutive_frames_required: 2

zones_file: ./demo/zones.json
mission_file: ./demo/mission.yaml

actions:
  ui_alert: true
  save_evidence: true
  create_response_task: true
  webhook:
    enabled: false
    url_env: RAILGUARD_WEBHOOK_URL
```

zones.json

```
{
  "source_id": "demo_platform_01",
  "reference_width": 1280,
  "reference_height": 720,
  "zones": [
    {
      "zone_id": "platform_edge",
      "name": "Platform Edge",
      "type": "RESTRICTED",
```

```

    "points_normalized": [[0.05,0.72],[0.95,0.68],[0.98,0.98],[0.02,0.98]]
  }
]
}

```

mission.yaml

```

mission_id: platform_intrusion_v1
name: Platform Edge Intrusion
objects: [person, train]
rules:
  - rule_id: person_enters_platform_edge
    event_type: RESTRICTED_ZONE_INTRUSION
    when:
      transition: OUTSIDE_TO_INSIDE
      object_class: person
      zone_id: platform_edge
      severity: HIGH
    escalate:
      - if: train_visible
        severity: CRITICAL
      cooldown_seconds: 5
    actions: [save_evidence, alert_ui, create_response_task]

```

Appendix C Core Algorithm Pseudocode

C.1 Frame Processing Loop

End-to-end loop

```

for packet in video_source:
    detections = detector.infer(packet.image)
    tracks = tracker.update(detections, packet.image)
    scene = context_builder.build(packet, tracks, zones)

    transitions = event_engine.update_track_states(scene)
    for transition in transitions:
        matching_rules = policy_engine.match(transition, scene)
        for rule in matching_rules:
            if dedupe.allow(rule, transition.track_id, packet.timestamp_ms):
                event = event_factory.create(rule, transition, scene, packet)
                event.evidence = evidence_service.save(event, packet.image, scene)
                event_store.insert(event)
                for action in action_registry.for_rule(rule):
                    result = action.execute(event)
                    event_store.append_action(event.event_id, result)
                event_bus.publish(event)

    ui_stream.publish(packet, tracks, scene, recent_events)

```

C.2 Hysteresis and State Transition

Jitter-resistant zone transition

```

inside_now = zone.contains(track.foot_point)
state = state_by_rule_track[(rule.id, track.id)]

if inside_now:
    state.inside_frames += 1
    state.outside_frames = 0
else:

```

```
state.outside_frames += 1
state.inside_frames = 0

entered = (
    state.previous_inside is False
    and state.inside_frames >= config.consecutive_frames_required
)
exited = (
    state.previous_inside is True
    and state.outside_frames >= config.consecutive_frames_required
)

if entered:
    emit_transition('OUTSIDE_TO_INSIDE')
if exited:
    emit_transition('INSIDE_TO_OUTSIDE')

state.previous_inside = inside_now
```

C.3 Severity Evaluation

P0 severity logic

```
severity = rule.base_severity
if scene.train_visible and rule.escalates_on_train:
    severity = 'CRITICAL'
return severity
```

C.4 Memory Query Routing

Deterministic event-memory Q&A

```
intent, params = intent_parser.parse(question)
if intent == 'MOST_CRITICAL':
    return event_store.most_critical(run_id)
if intent == 'COUNT_ZONE_ENTRIES':
    return event_store.count_intrusions(run_id, params.zone_id)
if intent == 'LAST_SEEN':
    return event_store.last_event_for_track(run_id, params.track_id)
return QueryResult.unsupported(supported_examples)
```

Appendix D Definition of Done

D.1 Product Done

Area	Done when
Mission	Preset and compiled rule visible.
Tracking	Stable person ID in demo.
Zone	Correct visible polygon and entry detection.
Incident	One event per entry with correct severity.
Evidence	Snapshot opens from timeline.
Action	Alert and response task logged.

Area	Done when
Memory	Five supported queries work.
Report	Generated from event database.
Demo	Five successful full runs.

D.2 Engineering Done

- Fresh environment setup follows README.
- No secrets or large generated files in Git.
- Model and third-party licenses are acknowledged.
- Core unit tests pass.
- Errors are visible and do not silently fail.
- Configuration used for demo is committed.
- Repository link and demo video link are verified in incognito/private mode.

D.3 Final Handoff Note

Build this in order

Do not polish the interface or add more use cases until a person crossing the zone creates one evidence-backed critical incident and that incident can be queried and reported. That loop is the product.

CORTEX RAILGUARD

Mission-driven visual safety. Local intelligence. Evidence-backed action.